

# Adam

CLASS torch.optim.Adam(*params*, *lr*=0.001, *betas*=(0.9, 0.999), *eps*=1e-08, *weight\_decay*=0, *amsgrad*=False, \*, *foreach*=None, *maximize*=False, *capturable*=False, *differentiable*=False, *fused*=None) [SOURCE]

Implements Adam algorithm.

```
input :  $\gamma$  (lr),  $\beta_1, \beta_2$  (betas),  $\theta_0$  (params),  $f(\theta)$  (objective)
         $\lambda$  (weight decay), amsgrad, maximize,  $\epsilon$  (epsilon)
initialize :  $m_0 \leftarrow 0$  ( first moment),  $v_0 \leftarrow 0$  (second moment),  $\widehat{v}_0^{max} \leftarrow 0$ 

for  $t = 1$  to ... do
    if maximize :
         $g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$ 
    else
         $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ 
    if  $\lambda \neq 0$ 
         $g_t \leftarrow g_t + \lambda \theta_{t-1}$ 
     $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
     $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
     $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
     $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
    if amsgrad
         $\widehat{v}_t^{max} \leftarrow \max(\widehat{v}_{t-1}^{max}, \widehat{v}_t)$ 
         $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t^{max}} + \epsilon)$ 
    else
         $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ 

return  $\theta_t$ 
```

For further details regarding the algorithm we refer to [Adam: A Method for Stochastic Optimization](#).

### Parameters

- **params** (*iterable*) – iterable of parameters or named\_parameters to optimize or iterable of dicts defining parameter groups. When using named\_parameters, all parameters in all groups should be named
- **lr** (*float*, *Tensor*, *optional*) – learning rate (default: 1e-3). A tensor LR is not yet supported for all our implementations. Please use a float LR if you are not also specifying fused=True or capturable=True.
- **betas** (*Tuple[*float*, *float*]*, *optional*) – coefficients used for computing running averages of gradient and its square (default: (0.9, 0.999))
- **eps** (*float*, *optional*) – term added to the denominator to improve numerical stability (default: 1e-8)
- **weight\_decay** (*float*, *optional*) – weight decay (L2 penalty) (default: 0)
- **amsgrad** (*bool*, *optional*) – whether to use the AMSGrad variant of this algorithm from the paper [On the Convergence of Adam and Beyond](#) (default: False)
- **foreach** (*bool*, *optional*) – whether foreach implementation of optimizer is used. If unspecified by the user (so foreach is None), we will try to use foreach over the for-loop implementation on CUDA, since it is usually significantly more performant. Note that the foreach implementation uses ~ sizeof(params) more peak memory than the for-loop version due to the intermediates being a tensorlist vs just one tensor. If memory is prohibitive, batch fewer parameters through the optimizer at a time or switch this flag to False (default: None)
- **maximize** (*bool*, *optional*) – maximize the objective with respect to the params, instead of minimizing (default: False)
- **capturable** (*bool*, *optional*) – whether this instance is safe to capture in a CUDA graph. Passing True can impair ungraphed performance, so if you don’t intend to graph capture this instance, leave it False (default: False)
- **differentiable** (*bool*, *optional*) – whether autograd should occur through the optimizer step in training. Otherwise, the step() function runs in a torch.no\_grad() context. Setting to True can impair performance, so leave it False if you don’t intend to run autograd through this instance (default: False)
- **fused** (*bool*, *optional*) – whether the fused implementation is used. Currently, *torch.float64*, *torch.float32*, *torch.float16*, and *torch.bfloat16* are supported. (default: None)

• NOTE

The foreach and fused implementations are typically faster than the for-loop, single-tensor implementation, with fused being theoretically fastest with both vertical and horizontal fusion. As such, if the user has not specified either flag (i.e., when foreach = fused = None), we will attempt defaulting to the foreach implementation when the tensors are all on CUDA. Why not fused? Since the fused implementation is relatively new, we want to give it sufficient bake-in time. To specify fused, pass True for fused. To force running the for-loop implementation, pass False for either foreach or fused.

• NOTE

A prototype implementation of Adam and AdamW for MPS supports `torch.float32` and `torch.float16`.

add\_param\_group(*param\_group*) [\[SOURCE\]](#)

Add a param group to the `Optimizer`’s `param_groups`.

This can be useful when fine tuning a pre-trained network as frozen layers can be made trainable and added to the `Optimizer` as training progresses.

**Parameters**

**param\_group** (*dict*) – Specifies what Tensors should be optimized along with group specific optimization options.

load\_state\_dict(*state\_dict*) [\[SOURCE\]](#)

Load the optimizer state.

**Parameters**

**state\_dict** (*dict*) – optimizer state. Should be an object returned from a call to `state_dict()` .

• NOTE

The names of the parameters (if they exist under the “param\_names” key of each param group in `state_dict()` ) will not affect the loading process. To use the parameters’ names for custom cases (such as when the parameters in the loaded state dict differ from those initialized in the optimizer), a custom `register_load_state_dict_pre_hook` should be implemented to adapt the loaded dict accordingly. If `param_names` exist in loaded state dict `param_groups` they will be saved and override the current names, if present, in the optimizer state. If they do not exist in loaded state dict, the optimizer `param_names` will remain unchanged.

register\_load\_state\_dict\_post\_hook(*hook*, *prepend=False*) [\[SOURCE\]](#)

Register a load\_state\_dict post-hook which will be called after `load_state_dict()` is called. It should have the following signature:

hook(optimizer) -> None

The `optimizer` argument is the optimizer instance being used.

The hook will be called with argument `self` after calling `load_state_dict` on `self` . The registered hook can be used to perform post-processing after `load_state_dict` has loaded the `state_dict` .

**Parameters**

- **hook** (*Callable*) – The user defined hook to be registered.
- **prepend** (*bool*) – If True, the provided post `hook` will be fired before all the already registered post-hooks on `load_state_dict` . Otherwise, the provided `hook` will be fired after all the already registered post-hooks. (default: False)

**Returns**

a handle that can be used to remove the added hook by calling `handle.remove()`

**Return type**

`torch.utils.hooks.RemoveableHandle`

register\_load\_state\_dict\_pre\_hook(*hook*, *prepend=False*) [\[SOURCE\]](#)

Register a load\_state\_dict pre-hook which will be called before `load_state_dict()` is called. It should have the following signature:

hook(optimizer, state\_dict) -> state\_dict or None

The `optimizer` argument is the optimizer instance being used and the `state_dict` argument is a shallow copy of the `state_dict` the user passed in to `load_state_dict` . The hook may modify the state\_dict inplace or optionally return a new one. If a state\_dict is returned, it will be used to be loaded into the optimizer.

The hook will be called with argument `self` and `state_dict` before calling `load_state_dict` on `self` . The registered hook can be used to perform pre-processing before the `load_state_dict` call is made.

**Parameters**

- **hook** (*Callable*) – The user defined hook to be registered.
- **prepend** (*bool*) – If True, the provided pre `hook` will be fired before all the already registered pre-hooks on `load_state_dict` . Otherwise, the provided `hook` will be fired after all the already registered pre-hooks. (default: False)

**Returns**

a handle that can be used to remove the added hook by calling `handle.remove()`

**Return type**

`torch.utils.hooks.RemoveableHandle`

register\_state\_dict\_post\_hook(*hook*, *prepend=False*) [\[SOURCE\]](#)

Register a state dict post-hook which will be called after `state_dict()` is called.

It should have the following signature:

```
hook(optimizer, state_dict) -> state_dict or None
```

The hook will be called with arguments `self` and `state_dict` after generating a `state_dict` on `self`. The hook may modify the `state_dict` inplace or optionally return a new one. The registered hook can be used to perform post-processing on the `state_dict` before it is returned.

Parameters

- **hook** (*Callable*) – The user defined hook to be registered.
- **prepend** (*bool*) – If True, the provided post `hook` will be fired before all the already registered post-hooks on `state_dict`. Otherwise, the provided `hook` will be fired after all the already registered post-hooks. (default: False)

Returns

a handle that can be used to remove the added hook by calling `handle.remove()`

Return type

`torch.utils.hooks.RemoveableHandle`

`register_state_dict_pre_hook(hook, prepend=False)` [\[SOURCE\]](#)

Register a state dict pre-hook which will be called before `state_dict()` is called.

It should have the following signature:

```
hook(optimizer) -> None
```

The `optimizer` argument is the optimizer instance being used. The hook will be called with argument `self` before calling `state_dict` on `self`. The registered hook can be used to perform pre-processing before the `state_dict` call is made.

Parameters

- **hook** (*Callable*) – The user defined hook to be registered.
- **prepend** (*bool*) – If True, the provided pre `hook` will be fired before all the already registered pre-hooks on `state_dict`. Otherwise, the provided `hook` will be fired after all the already registered pre-hooks. (default: False)

Returns

a handle that can be used to remove the added hook by calling `handle.remove()`

Return type

`torch.utils.hooks.RemoveableHandle`

`register_step_post_hook(hook)` [\[SOURCE\]](#)

Register an optimizer step post hook which will be called after optimizer step.

It should have the following signature:

```
hook(optimizer, args, kwargs) -> None
```

The `optimizer` argument is the optimizer instance being used.

Parameters

**hook** (*Callable*) – The user defined hook to be registered.

Returns

a handle that can be used to remove the added hook by calling `handle.remove()`

Return type

`torch.utils.hooks.RemovableHandle`

`register_step_pre_hook(hook)` [\[SOURCE\]](#)

Register an optimizer step pre hook which will be called before optimizer step.

It should have the following signature:

```
hook(optimizer, args, kwargs) -> None or modified args and kwargs
```

The `optimizer` argument is the optimizer instance being used. If args and kwargs are modified by the pre-hook, then the transformed values are returned as a tuple containing the `new_args` and `new_kwargs`.

Parameters

**hook** (*Callable*) – The user defined hook to be registered.

Returns

a handle that can be used to remove the added hook by calling `handle.remove()`

Return type

`torch.utils.hooks.RemovableHandle`

`state_dict()` [\[SOURCE\]](#)

Return the state of the optimizer as a `dict`.

It contains two entries:

- state**: a Dict holding current optimization state. Its content differs between optimizer classes, but some common characteristics hold. For example, state is saved per parameter, and the parameter itself is NOT saved. `state` is a Dictionary mapping parameter ids to a Dict with state corresponding to each parameter.
- param\_groups**: a List containing all parameter groups where each parameter group is a Dict. Each parameter group contains metadata specific to the optimizer, such as learning rate and weight decay, as well as a List of parameter IDs of the parameters in the group. If a param group was initialized with `named_parameters()` the names content will also be saved in the state dict.

NOTE: The parameter IDs may look like indices but they are just IDs associating state with param\_group. When loading from a `state_dict`, the optimizer will zip the `param_group params` (int IDs) and the optimizer `param_groups` (actual `nn.Parameter` s) in order to match state WITHOUT additional verification.

A returned state dict might look something like:

```
{
  'state': {
    0: {'momentum_buffer': tensor(...), ...},
    1: {'momentum_buffer': tensor(...), ...},
    2: {'momentum_buffer': tensor(...), ...},
    3: {'momentum_buffer': tensor(...), ...}
  },
  'param_groups': [
    {
      'lr': 0.01,
      'weight_decay': 0,
      ...
      'params': [0]
      'param_names' ['param0'] (optional)
    },
    {
      'lr': 0.001,
      'weight_decay': 0.5,
      ...
      'params': [1, 2, 3]
      'param_names': ['param1', 'layer.weight', 'layer.bias'] (optional)
    }
  ]
}
```

Return type

`Dict[str, Any]`

`step(closure=None)` [\[SOURCE\]](#)

Perform a single optimization step.

Parameters

**closure** (*Callable, optional*) – A closure that reevaluates the model and returns the loss.

`zero_grad(set_to_none=True)` [\[SOURCE\]](#)

Reset the gradients of all optimized `torch.Tensor` s.

Parameters

**set\_to\_none** (*bool*) – instead of setting to zero, set the grads to None. This will in general have lower memory footprint, and can modestly improve performance. However, it changes certain behaviors. For example: 1. When the user tries to access a gradient and perform manual ops on it, a None attribute or a Tensor full of 0s will behave differently. 2. If the user requests `zero_grad(set_to_none=True)` followed by a backward pass, `.grad` s are guaranteed to be None for params that did not receive a gradient. 3. `torch.optim` optimizers have a different behavior if the gradient is 0 or None (in one case it does the step with a gradient of 0 and in the other it skips the step altogether).